

Белорусский Государственный Университет
Факультет прикладной математики и информатики

Компьютерная графика

Кулинкович Виктория Александровна

9 лекций,

8 лабораторных занятий

Алгоритмы сжатия графической информации

- Сжатие графической информации. Избыточность данных.
- Критерии сравнения алгоритмов сжатия.
- Алгоритм кодирования длины повторения (RLE).
- Словарные алгоритмы: алгоритм LZ77, алгоритм LZW.
- Алгоритмы статистического кодирования: Алгоритм Хаффмана. Арифметическое кодирование.
- Необходимость сжатия с потерями. Оценка потерь. Алгоритм сжатия изображений JPEG.



Растровые графические форматы

BMP GIF PNG JPEG TIFF PCX TGA IMG

Чем отличаются:
- структура файла
- глубина поддерживаемой палитры
- тип сжатия
- поддержка различных цветовых моделей
- поддержка прозрачности
- поддержка анимации
- и т.д.

Глубина цвета

Глубина цвета (или битовая глубина) – это количество градаций, используемых для представления каждого пикселя растрового изображения. Указывается в количестве бит на пиксель или бит на канал для данной цветовой модели.

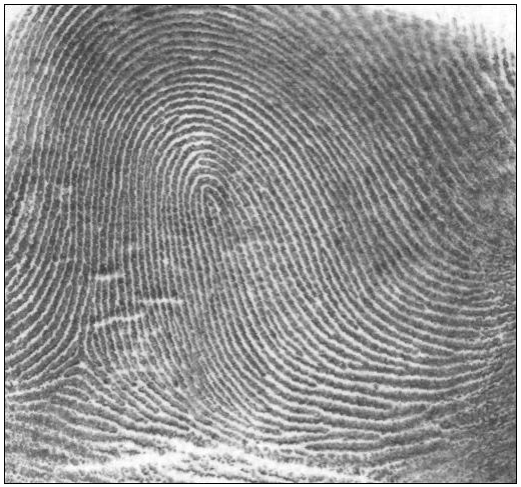
Классификация изображений в зависимости от глубины цвета:

- Бинарные изображения (двухцветные): на каждый пиксель отводится один бит; обычно это черно-белые изображения.



Глубина цвета

- Grayscale-изображения (в градациях серого). Каждый пиксель имеет яркость в диапазоне от 0 до 255.



- Индексированные 8-битные изображения.

- Полноцветные изображения:
Цвет каждого пикселя задается 3 каналами, каждый по одному байту (например, (R,G,B).





Сжатие изображений

Сжатие данных – алгоритмическое преобразование данных, производимое с целью уменьшения занимаемого ими объёма.

Под **сжатием изображений** понимают применение алгоритмов сжатия данных к изображениям, хранящимся в цифровом виде.

Сжатие изображений обусловлено тремя особенностями:

- Большой объем памяти, занимаемой растровыми изображениями.

- Особенность человеческого восприятия: человеческое зрение при анализе изображения оперирует контурами, общим переходом цветов и сравнительно нечувствительно к малым изменениям в изображении.

- Особенности структуры изображения: изображение, в отличие, например, от текста, обладает избыточностью в двухмерном измерении.

Избыточность данных

В растровых изображениях есть два типа избыточности:

- В случайном изображении некоторые цвета преобладают, а другие встречаются редко.

- Избыточность в результате корреляции пикселей.



Задача выбора алгоритма сжатия

Пример классификация изображений с целью выбора алгоритма сжатия:

Класс 1. Изображения с небольшим количеством цветов (4-16) и большими областями, заполненными одним цветом. Плавные переходы цветов отсутствуют. Примеры: деловая графика — гистограммы, диаграммы, графики и т.п.
Класс 2. Изображения, с плавными переходами цветов, построенные на компьютере. Примеры: графика презентаций, эскизные модели в САПР.
Класс 3. Фотореалистичные изображения. Пример: отсканированные фотографии.
Класс 4. Фотореалистичные изображения с наложением деловой графики. Пример: реклама.

Достаточно сложной и интересной задачей является поиск наилучшего алгоритма для конкретного класса изображений.

Критерии сравнения алгоритмов сжатия

- Худший, средний и лучший коэффициенты сжатия.

$$коэффициент\,сжатия = \frac{размер\,выходного\,файла}{размер\,входного\,файла}$$

- Класс изображений, на который ориентирован алгоритм.

- Симметричность (соотношение ресурсоёмкости процесса сжатия и обратного процесса восстановления).

- Потеря качества.

- Характерные особенности алгоритма и изображений, к которым его применяют.

Сжатие с потерями и без потерь

A называется алгоритмом **сжатия без потерь** (англ. *Lossless compression*), если существует алгоритм $\mathbf{A^{-1}}$ (обратный к **A**) такой, что для любого изображения **I** верно:

$$\mathbf{A(I) = I_1 \text{ и } A^{-1}(I_1) = I}$$

A называется алгоритмом **сжатия с потерями** (англ. *Lossy compression*), если он не обеспечивает возможность точного восстановления исходного изображения. Если парный к **A** алгоритм, обеспечивающий примерное восстановление, обозначить как $\mathbf{A^*}$, тогда для изображения **I** будет верно:

$$\mathbf{A(I) = I_1,\ A^*(I_1) = I_2}$$

и при этом полученное восстановленное изображение $\mathbf{I_2}$ не обязательно точно совпадает с **I**. Пара **A**, $\mathbf{A^*}$ подбирается так, чтобы обеспечить большие коэффициенты сжатия и тем не менее сохранить визуальное качество, т.е. добиться минимальной разницы в восприятии между **I** и $\mathbf{I_2}$.

Сжатие изображений. Основные определения.

Кодер(**компрессор**) – программа, которая сжимает исходный файл и создает на выходе файл со сжатыми данными, в которых мало избыточности.

Декодер (**декомпрессор**) – работает в обратном направлении.

Симметричное сжатие – сжатие, при котором и кодер, и декодер используют один и тот же базовый алгоритм, но используют его в противоположных направлениях.



Алгоритм RLE

RLE (*run-length encoding* – кодирование длин серий, или кодирование повторов) – алгоритм сжатия данных, заменяющий повторяющиеся символы (серии) на один символ и число его повторов.

Существует 2 варианта работы алгоритма – на битовом уровне и байтовом уровне.

Используется в следующих растровых форматах:
BMP (по желанию)
PCX (всегда)
TGA (по желанию)
IMG (всегда)
TIFF (по желанию)

Алгоритм рассчитан на деловую графику – изображения с большими областями повторяющегося цвета.



Алгоритм RLE

RLE - битовый уровень.

Данные представляются на уровне последовательности битов (например, черно-белое изображение).

Например: последовательности

0000111110001111111111

соответствует набор чисел количеств повторений

4 5 3 11



Алгоритм RLE

RLE - битовый уровень.

Однако, учитывая то, что числа, обозначающие количество повторений, также надо кодировать битами, считается, что каждое число повторений изменяется от 0 до 7 (т.е. можно закодировать ровно тремя битами), и они чередуются.

Например, последовательности

11111111111

будет сопоставлена сжатая последовательность

111 000 100

Последовательность, состоящая из

21 единицы, 21 нуля, 3 единиц и 1 ноль

закодируется так

111 000 111 000 111 111 000 111 000 111 011 001

Алгоритм RLE

RLE - байтовый уровень.
Данные представляются на уровне последовательности байтов (например, grayscale изображение).

Изображение вытягивается в цепочку байт по строкам растра и далее этот входной поток разбивается на байты (буквы) и повторяющиеся байты кодируются в двухбайтную пару *(счетчик, буква).*
Пример: AABBBCDAA кодируем (2A) (3B) (1C) (1D) (2A)

Признаком счетчика служат, например, единицы в двух верхних битах считанного файла; соответственно оставшиеся 6 бит расходуются на счетчик, который может принимать значения от 1 до 64. Таким образом, строка из 64 повторяющихся байтов превращается в два байта, т.е. сжимается в 32 раза.

Алгоритм RLE

Модификация алгоритма.

Для улучшения результатов работы для входных данных используется предварительная обработка.

Преобразование Барроуза-Уилера (англ. *Burrows-Wheeler transform* - **BWT**) – алгоритм, используемый для предварительной обработки данных перед сжатием, разработанный для улучшения эффективности последующего кодирования. Преобразование Барроуза-Уилера меняет порядок символов во входной строке таким образом, что повторяющиеся подстроки образуют на выходе идущие подряд последовательности одинаковых символов.



Алгоритм RLE

Преобразование Барроуза-Уилера.

Преобразование выполняется в три этапа:

1. Составляется таблица всех циклических перестановок входной строки.
2. Производится лексикографическая (в алфавитном порядке) сортировка строк таблицы.
3. В качестве выходной строки выбирается последний столбец таблицы преобразования и номер строки, совпадающей с исходной.

Алгоритм RLE

Преобразование Барроуза-Уилера. Пример.

Трансформация			
Вход	**Все Перестановки**	**Сортировка Строк**	**Выход**
.BANANA.	.BANANA. ..BANANA A..BANAN NA..BANA ANA..BAN NANA..BA ANANA..B BANANA..	ANANA..B ANA..BAN A..BANAN BANANA.. NANA..BA NA..BANA .BANANA. ..BANANA	BNN.AA.A

Алгоритм RLE

Преимущества:

простота и скорость работы (как скорость кодирования, так и скорость декодирования)

Недостатки:

неэффективность на неповторяющихся наборах символов. Использование специальных перестановок повышает эффективность алгоритма, но при этом сильно увеличивает время работы.

Характеристики алгоритма RLE

Коэффициенты сжатия:
Первый вариант: 32, 2, 0,5. Второй вариант: 64, 3, 128/129
(Лучший, средний, худший коэффициенты)

Класс изображений: Ориентирован алгоритм на изображения с небольшим количеством повторяющихся цветов: деловую и научную графику.

Симметричность: симметричен.

Характерные особенности: К положительным сторонам алгоритма можно отнести только то, что он не требует дополнительной памяти при архивации и разархивации, а также быстро работает. Интересная особенность группового кодирования состоит в том, что степень архивации для некоторых изображений может быть существенно повышена всего лишь за счет изменения порядка цветов в палитре изображения.



Алгоритм LZ77

Словарный метод сжатия информации.

Разработан в 1970 году и опубликован в 1977 году учеными Абрахамом Лемпелем и Якобом Зивом.

Основная идея состоит в использовании ранее прочитанной последовательности входных данных в качестве динамического словаря.



Алгоритм LZ77

Для входного потока данных кодер создает скользящее окно, содержащее последовательность символов – **буфер**, или **словарь**.

Буфер разбивается на 2 части – слева **буфер поиска** и справа **буфер предварительного просмотра**.

Буфер поиска содержит символы, которые поступили недавно и уже были закодированы (длина – несколько тысяч байт).

В **буфере предварительного просмотра** содержится текст, который надо закодировать (длина – несколько десятков байт).

Алгоритм LZ77

Алгоритм работы кодера:

1. Кодер просматривает буфер поиска справа налево и ищет в нем первое появление текущего символа из буфера предварительного просмотра.
2. Кодер определяет, сколько совпадающих символов следует за найденным символом и текущим, и запоминает длину ***L*** найденной совпадающей подпоследовательности и смещение ***S*** найденного символа относительно текущего.
3. Кодер продолжает поиск по буферу поиска, пытаясь найти более длинные совпадающие подпоследовательности, обновляя ***L*** и ***S***.
4. Если найдены 2 совпадающие подпоследовательности одинаковой длины, то кодер выбирает наиболее удаленную из них.
5. В выходной поток записывается метка < ***S***; ***L***; очередной необработанный символ в буфере предварительного просмотра>
6. Буфер сдвигается вправо на число позиций, равное ***L+1***.
7. Если обратный поиск не обнаружил ни одной совпадающей подпоследовательности, то записывается метка со смещением 0, длиной 0 и текущим символом.

Алгоритм LZ77

Пример работы.
Входная последовательность «абракадабра»
Размер буфера поиска: 20. Размер буфера предварительного просмотра: 4

Шаг	Буфер	Макс.совпавшая подпосл-ть	Метка
1	«»+«абра»	-	<0,0,а>
2	«а»+«брак»	-	<0,0,б>
3	«аб»+«рака»	-	<0,0,р>
4	«абр»+«акад»	а	<3,1,к>
5	«абрак»+«адаб»	а	<5,1,д>
6	«абракад»+«абра»	абра	<7,4,->

Алгоритм LZ77

Алгоритм работы декодера:

1. Декодер последовательно строит свой буфер поиска
2. Декодер считывает метку, находит совпадение в своем буфере, записывает совпадающую подпоследовательность, а затем символ из третьего поля.

<0,0,а> → «а»
<0,0,б> → «аб»
<0,0,р> → «абр»
<3,1,к> → «абрак»
<5,1,д> → «абракад»
<7,4,-> → «абракадабра»

Алгоритм LZ77

Преимущества:

не вносит искажений в исходные данные;
не требует хранения дополнительной информации;

Недостатки:

алгоритм основан на предположении, что похожие образцы сживаемых данных находятся недалеко друг от друга; если содержимое файла этому условию не удовлетворяет, то он будет сжиматься плохо;
ограниченные размеры буфера; Увеличение размеров буфера могло бы, с одной стороны, улучшить коэффициент сжатия, но с другой стороны, увеличивает время поиска.



Алгоритм LZ78

Словарный метод сжатия информации.

Идея: алгоритм в явном виде использует словарный подход, генерируя временный словарь во время кодирования и декодирования.

Отличие от LZ77: LZ78 не использует скользящее окно, он хранит словарь из уже просмотренных фраз.



Алгоритм LZ78

Изначально словарь пуст, а алгоритм пытается закодировать первый символ. На каждой итерации алгоритм ищет в словаре максимальную совпадающую подпоследовательность. Метки этого алгоритма будут состоять уже из двух полей – индекса в словаре самой длинной найденной подпоследовательности и символа, который идет за этой подпоследовательностью в обрабатываемых данных. При этом после кодирования такой пары подпоследовательность с приписанным символом добавляется в словарь, а алгоритм продолжает кодирование со следующего символа.



Алгоритм LZ78

Одна из главных проблем LZ78 — размер словаря. Он ничем не ограничен и может достигать огромных размеров на больших объемах входных данных. В различных модификациях установлены лимиты на размер словаря: при достижении определенного лимита словарь может «замораживаться» (в него перестают добавляться новые слова), из словаря могут удалять менее используемые или самые старые слова и так далее.

Хорошей структурой для организации словаря является дерево.



Алгоритм LZW

Словарный метод сжатия информации.

Модификация алгоритма LZ78.

Разработан в 1984 году Терри Уелчем.

Используется в следующих растровых форматах:
GIF (всегда)
TIFF (по желанию)



Сжатие изображений. LZW

На самом начальном этапе словарь заполняется всеми символами входного алфавита.

В процессе сжатия отыскивается наиболее длинная цепочка, уже записанная в словарь.

Каждый раз, когда новая цепочка элементов не найдена в словаре, она добавляется в словарь; при этом записывается код цепочки, для которой есть совпадение со словарем.

Процесс сжатия выглядит достаточно просто. Мы считываем последовательно символы входного потока и проверяем, есть ли в созданной нами таблице строк такая строка. Если строка есть, то мы считываем следующий символ, а если строки нет, то мы заносим в поток код для предыдущей найденной строки, заносим строку в таблицу и начинаем поиск снова.

Особенность LZW заключается в том, что для декомпрессии нам не надо сохранять таблицу строк в файл для распаковки. Алгоритм построен таким образом, что мы в состоянии восстановить таблицу строк, пользуясь только потоком кодов. *Мы знаем, что для каждого кода надо добавлять в таблицу строку, состоящую из уже присутствующей там строки и символа, в которого начинается следующая строка в потоке.*



Характеристики алгоритма LZW

Коэффициенты сжатия: ~ 1000, 4, 5/7 (лучший, средний, худший коэффициенты).
(Сжатие в 1000 раз достигается только на одноцветных изображениях размером кратным примерно 7 Мб)

Класс изображений: Ориентирован на 8-битные изображения.

Симметричность: Почти симметричен, при условии оптимальной реализации операции поиска строки в таблице.

Характерные особенности: Ситуация, когда алгоритм увеличивает изображение, встречается крайне редко. LZW универсален – именно его варианты используются в обычных архиваторах.

Алгоритм Хаффмана

Один из классических алгоритмов, известных с 60-х годов. Относится к классу статистических методов: для кодирования использует статистические свойства сжимаемых данных.

Алгоритм Хаффмана использует особый вид представления элементов – **префиксный код**. **Префиксный код** – это код переменной длины, обладающий свойством префикса: менее короткие коды не совпадают с префиксом (начальной частью) более длинных. Такой код позволяет осуществлять взаимно-однозначное кодирование.

Идея алгоритма: использовать частоту появления одинаковых байт в изображении. Сопоставляет символам входного потока, которые встречаются большее число раз, цепочку бит меньшей длины. И, напротив, встречающимся редко – цепочку большей длины. Для сбора статистики необходим дополнительный проход по изображению.

Декодирование осуществляется прямой заменой кода на соответствующий элемент. Алгоритм требует записи в файл таблицы соответствия кодируемых символов и кодирующих цепочек.



Алгоритм Хаффмана

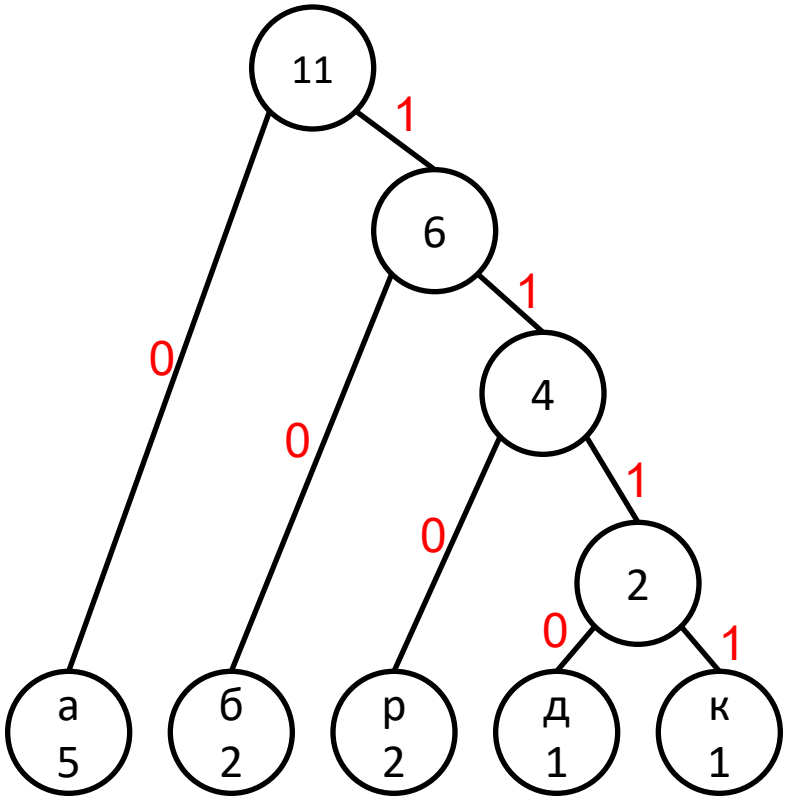
Алгоритм работы кодера:

1. Составляется список кодируемых символов, при этом один символ рассматривается как узлы двоичного дерева, с весом, равным частоте появления символа во входном потоке.
2. Символы упорядочиваются по убыванию частот.
3. Из списка выбираются два символа с наименьшими весами.
4. Формируется новый узел, с весом, равным сумме весов двух выбранных узлов, и становится их предком.
5. Если свободных узлов более одного, то переходим к пункту 3.
6. Код соответствующего символа – путь от соответсвующего узла с символом к вершине.

Алгоритм Хаффмана

Пример работы кодера:

1. Входной поток: «абракадабра»
2. Список входных символов: {а, б, д, к, р}. Частоты: {5, 2, 1, 1, 2}.





Характеристики алгоритма Хаффмана

Коэффициенты компрессии: 8, 1.5, 1
(Лучший, средний, худший коэффициенты).

Класс изображений: Практически не применяется к изображениям в чистом виде. Обычно используется как один из этапов компрессии в более сложных схемах.

Симметричность: 2 (за счет того, что требует двух проходов по массиву сжимаемых данных).

Характерные особенности: Единственный алгоритм, который не увеличивает размера исходных данных в худшем случае (если не считать необходимости хранить таблицу перекодировки вместе с файлом).

LZ77 + Хаффман = Deflate → используется в PNG и ZIP!



Алгоритм Хаффмана с фиксированной таблицей CCITT Group 3

Используется при сжатии черно-белых изображений (один бит на пиксель). Полное название данного алгоритма CCITT Group 3. Это означает, что данный алгоритм был предложен третьей группой по стандартизации Международного Консультационного Комитета по Телеграфии и Телефонии (Consultative Committee International Telegraph and Telephone). Последовательности подряд идущих черных и белых точек в нем заменяются числом, равным их количеству. А этот ряд, уже в свою очередь, сжимается по Хаффману с фиксированной таблицей.

Характеристики алгоритма CCITT Group 3

Коэффициенты сжатия: лучший коэффициент стремится в пределе к 213.(3), средний 2, в худшем случае увеличивает файл в 5 раз.

Класс изображений: Двуцветные черно-белые изображения, в которых преобладают большие пространства, заполненные белым цветом.

Симметричность: почти симметричен.

Характерные особенности: Данный алгоритм чрезвычайно прост в реализации, быстр и может быть легко реализован аппаратно.



Сжатие изображений с потерями

Многие изображения практически сжимаются стандартными алгоритмами, хотя, на первый взгляд, обладали явной избыточностью. Это привело к созданию нового типа алгоритмов – сжимающих с потерей информации.

Как правило, коэффициент сжатия и, следовательно, степень потерь качества в них можно задавать. При этом достигается компромисс между размером и качеством изображений.



Сжатие изображений с потерями

Со временем стандартные алгоритмы сжатия, рассчитанные на текстовую информацию, перестали удовлетворять предъявляемым требованиям.

Отличия между текстовой и графической информацией.

1. Текст одномер, а изображения – двухмерны.
2. Огромное количество возможных значений точек полноцветного изображения.
3. Существуют методы, которые автоматически удаляют неважную информацию из растрового изображения.

Сжатие изображений с потерями

Оценка потерь

- root mean square – RMS

$$d(x,y) = \sqrt{\frac{\sum_{i=1,j=1}^{n,n}\left(x_{ij} - y_{ij}\right)^2}{n^2}}$$

- peak-to-peak signal-to-noise ratio – PSNR

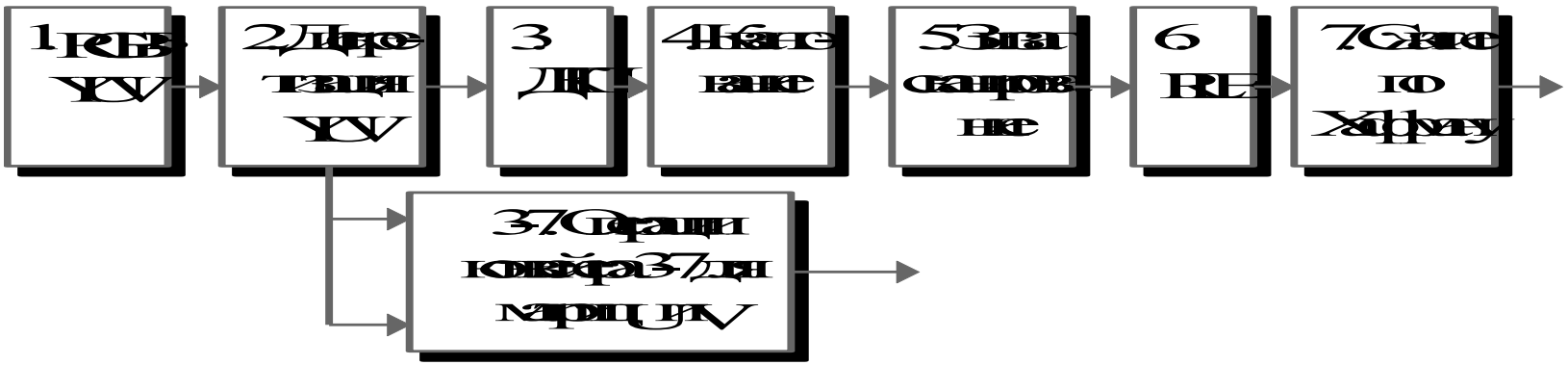
$$d(x,y) = 10 \cdot \log_{10} \frac{255^2 \cdot n^2}{\sum_{i=1,j=1}^{n,n}\left(x_{ij} - y_{ij}\right)^2}$$

- самой важной «мерой» оценки потерь является мнение наблюдателя, то есть визуальная оценка

Сжатие изображений с потерями

JPEG (Joint Photographic Expert Group) – один из самых новых и мощных алгоритмов сжатия с потерями.

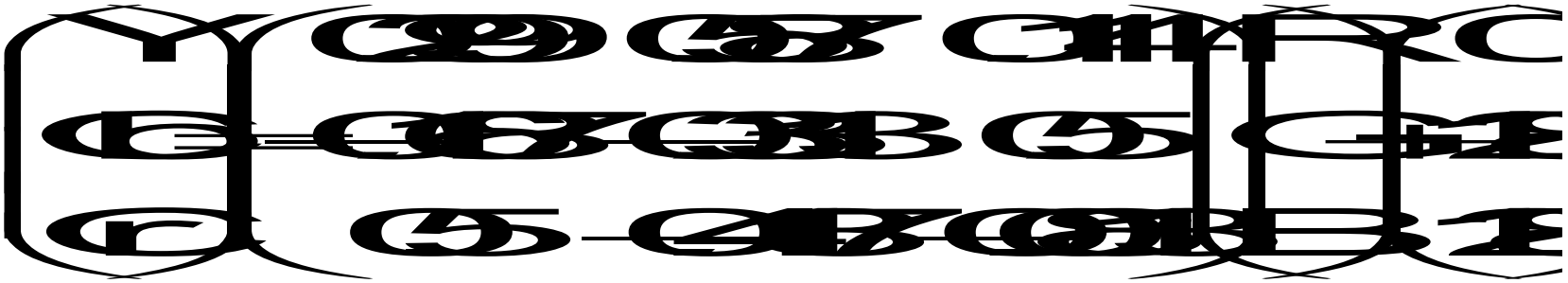
Конвейер операций, используемый в алгоритме JPEG.



JPEG

Шаг 1.

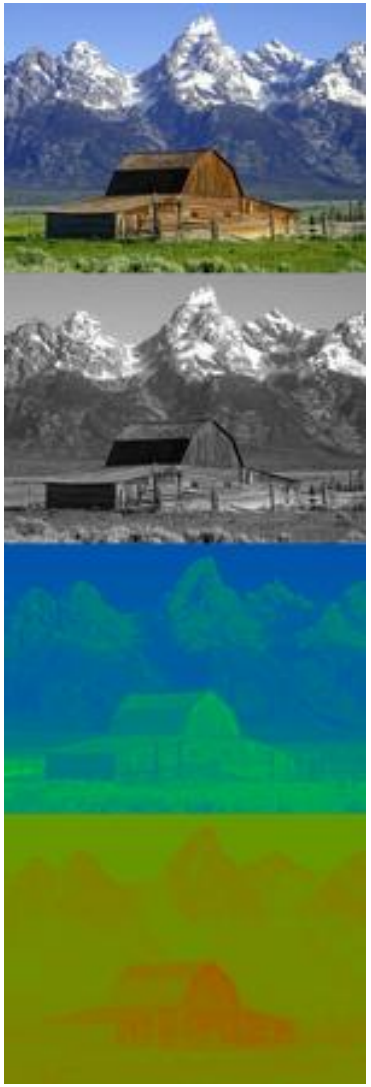
Изображение переводится из цветового пространства RGB в цветовое пространство YCrCb (YUV).



JPEG

Шаг 1.

Изображение переводится из цветового пространства RGB в цветовое пространство YCrCb (иногда называют YUV).



JPEG

Шаг 2.

Субдискретизация компонент цветности.

Большая часть визуальной информации, к которой наиболее чувствительны глаза человека, состоит из высокочастотных, полутоновых компонентов яркости (Y) цветового пространства YCbCr. Две других составляющих цветности (Cb и Cr) содержат высокочастотную цветовую информацию, к которой глаз человека менее чувствителен. Следовательно, определенная ее часть может быть отброшена. Например, в изображении размером 1000x1000 пикселей можно использовать яркости всех 1000x1000 пикселей, но только 500x500 пикселей для каждого компонента цветности. При таком представлении каждый пиксель цветности будет охватывать ту же область, что и (для яркости). В результате мы сохраним для каждого блока 2x2 всего блок 2x2 пикселя 6 пиксельных значений (4 значения яркости и по 1 значению для каждого из двух каналов цветности) вместо того, чтобы использовать 12 значений при обычном описании. Практика показала, что уменьшение объема данных на 50% почти незаметно отражается на качестве большинства изображений.

JPEG

Шаг 2.

Субдискретизация компонент цветности.

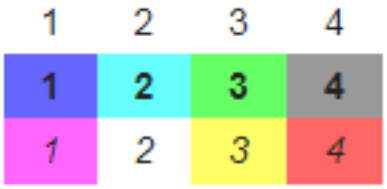
Стандарт JPEG предлагает несколько различных вариантов определения коэффициентов субдискретизации. Канал яркости Y всегда остается с полным разрешением. Каналы Cr и Cb дублируются на соседние пиксели (так называемое «укрупнение» пикселей).

Структура субдискретизации описывается как X:a:b.

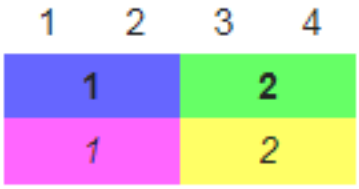
(X – ширина блока; a - число выборок в горизонтальном направлении в первой строке;

b - число выборок во второй строке)

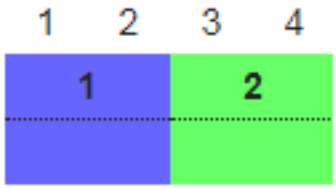
4:4:4 (без изменений)



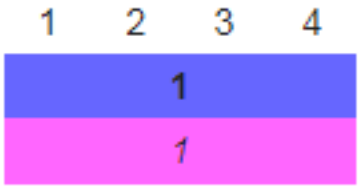
4:2:2



4:2:0



4:1:1



Компонента светимости Y остаётся неизменной!

JPEG

Шаг 3.

Применение дискретного косинус-преобразования (ДКП).

Пиксели каждой цветной компоненты собираются в блоки ***8x8*** , которые называются ***единицами данных***. Далее к каждой единице данных применяется ДКП, описываемое формулой:

$$G_{ij} = \frac{1}{\sqrt{2n}} C_i C_j \sum_{x=0}^{n-1} \sum_{y=0}^{n-1} p_{xy} \cos\left(\frac{(2y+1)j\pi}{2n}\right) \cos\left(\frac{(2x+1)i\pi}{2n}\right), \quad i, j = \overline{1, n}$$

$$C_i = \begin{cases} \frac{1}{\sqrt{2}}, i = 0 \\ 1, \quad i > 0 \end{cases}$$

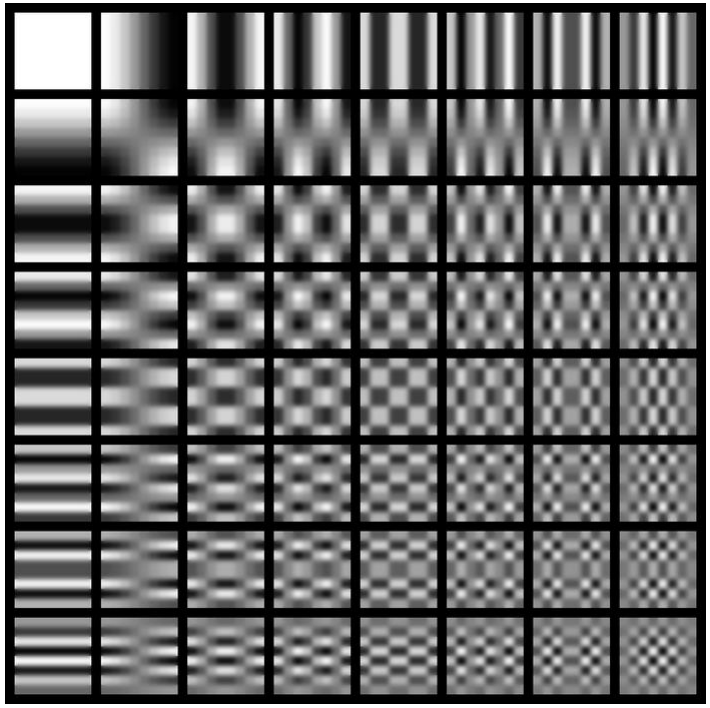
ДКП применяется для каждой единицы данных размера n×n=8×8.

В результате из каждой единицы данных получаем новую матрицу 8×8, которая содержит частотные коэффициенты; коэффициенты в левом верхнем углу соответствуют низкочастотной составляющей изображения, а в правом нижнем — высокочастотной.

JPEG

Шаг 3.

Применение дискретного косинус-преобразования (ДКП). Базовые функции.



JPEG

Шаг 4.

Квантование

Термин «квантование» при использовании в сжатии данных означает округление вещественных чисел до целых или преобразование целых чисел в меньшие целые.

Квантование осуществляется с помощью деления матрицы коэффициентов ДКП на так называемую матрицу квантования (МК).

$$Gq_{ij} = IntegerRound(\frac{G_{ij}}{q_{ij}})$$

В стандарт JPEG включены рекомендованные МК, построенные опытным путем. Матрицы для большего или меньшего коэффициентов сжатия получают путем умножения исходной матрицы на некоторое число gamma.

JPEG

Шаг 4.

Квантование. Рекомендуемая JPEG матрица квантования

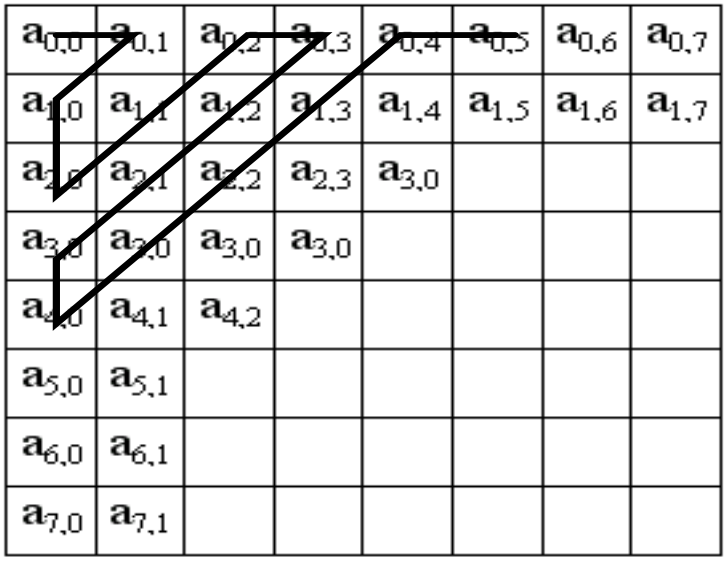
16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

JPEG

Шаг 5.

Зигзаг-упорядочивание.

Матрица 8x8 переводится в 64-элементный вектор при помощи «зигзаг»-сканирования, т.е. берутся элементы с индексами (0,0), (0,1), (1,0), (2,0)…



JPEG

Шаг 6.

Сжатие методом RLE

Полученный после зигзаг-сканирования вектор сжимается с помощью алгоритма группового кодирования (RLE).

Шаг 7.

Получившиеся пары сжимаются кодированием по Хаффману с фиксированной таблицей.



Характеристики алгоритма JPEG

Коэффициенты сжатия: 2-200 (Задается пользователем).

Класс изображений: Полноцветные 24 битные изображения или изображения в градациях серого без резких переходов цветов (фотографии).

Симметричность: 1.

Характерные особенности: В некоторых случаях, алгоритм создает «ореол» вокруг резких горизонтальных и вертикальных границ в изображении (эффект Гиббса). Кроме того, при высокой степени сжатия изображение распадается на блоки 8x8 пикселей.



Сравнение алгоритмов сжатия

Без потерь:
RLE — простые изображения с повторами (факс, PCX)
LZ77 — основа PNG и ZIP
LZW — GIF, TIFF
Хаффман — компонент внутри JPEG и ZIP
CCITT Group 3 — чёрно-белые документы, факс

С потерями:
JPEG — фотографии